Mini Project 1 Specification:
Server Redundancy Management System (Java)

COMP 370 - Software Engineering

Professor Majid Babaei

Team Information:

Simran Dhillon - 300211810 - simran.dhillon7@student.ufv.ca
Charvi Charvi - 300207650 - charvi@student.ufv.ca
Harjot Josan - 300211962 - harjot.josan@student.ufv.ca
Mohammad Salema - 300223224 - mohammad.salema@student.ufv.ca

Project Objectives

The main goal of our project was to build a system that keeps services running even when a hardware or software crash does occur. Our solution achieves this by splitting the system into independent Java processes (like Monitor, Client, and cluster of Servers) so that no single failure can take the whole thing down. We implemented a backup model where one server handles the workload while the others stay on standby. The ones on standby are ready to take over if something goes wrong and in order to automate this, we built a heartbeat system where servers constantly "check in" with the Monitor. If the Primary goes silent, the Monitor immediately spots the gap and promotes the backup with the lowest ID to take its place. This approach is necessary since it prevents "split-brain" scenarios where two servers try to act as the leader at once. On the front end, the Client doesn't just give up if a connection drops as it's programmed to ask the Monitor for the new Primary's location and reconnect automatically, making the transition nearly invisible to the user. We backed all of this up with a timestamped logging system, allowing us to track exactly how long detection and failover take during simulated crashes. By layering this automated logic over a multi-threaded networking core, we turned a group of basic Java programs into a reliable, coordinated cluster that stays online through primary crashes, backup failures, and network delays.

Descriptions of the Main Functions Implemented

The core of our Java project is built to focus on failover management and the center of this is the 'Monitor', which is the component that acts as the system's main authority. It listens on a fixed port and so the Monitor serves as the hub for all network traffic. It processes important message types such as 'REGISTER', 'HEARTBEAT', and any client queries for the current "Primary" server. So at startup the server starts the 'registerServer' logic in the Monitor which gives its unique ID, host, and port. All these details are then stored in a 'ConcurrentHashMap'. Once registered, the server maintains its "active" status through a dedicated 'HeartbeatSender' thread which also periodically pings the Monitor, which in turn updates a timestamp. If a server does not successfully check in within a certain timeout window, the Monitor then identifies the failure and removes that server so it's no longer active. If a server fails and it was the Primary, then the Monitor starts to find a new Primary from the remaining servers, so whichever remaining server has the lowest ID is given the role. This is done through a 'PROMOTE' message sent through a socket connection to the selected 'BackupServer'. Upon receiving this signal, our server process transitions from an idle state to an active one, getting the responsibility for all primary operations. The client and 'MonitorClient' classes first ask the Monitor to discover the current Primary and if a connection to that Primary is lost, the client does not crash and instead, it automatically queries the Monitor for the updated Primary address and reconnects. This makes the transition between servers effectively transparent to the end user. Finally, we also implemented a 'Logger' class which goes to record every registration, timeout, and promotion with precise timestamps, and provides us a complete audit trail. This allowed us to not only verify the system's logic but also helped us measure how quickly the system detects failures and recovers, ensuring the system meets the criteria well.

Requirements (FR/NFR) and Traceability to Design/Implementation

In this project, we first listed the main functional and non-functional requirements to guide the design of our Server Redundancy Management System (SRMS). These requirements helped make sure the system stays available even when failures happen.

Functional Requirements (FR)

- The system allows clients to send requests and receive responses from the current primary server. This is done through socket communication between the Client and Server.
- Only one server acts as the primary at a time, while the others stay as backup servers ready to take over.
- Servers send heartbeat messages regularly so the monitor can track which servers are still running.
- If the primary server crashes, the monitor detects missed heartbeats and promotes a backup server (lowest ID) to become the new Primary.

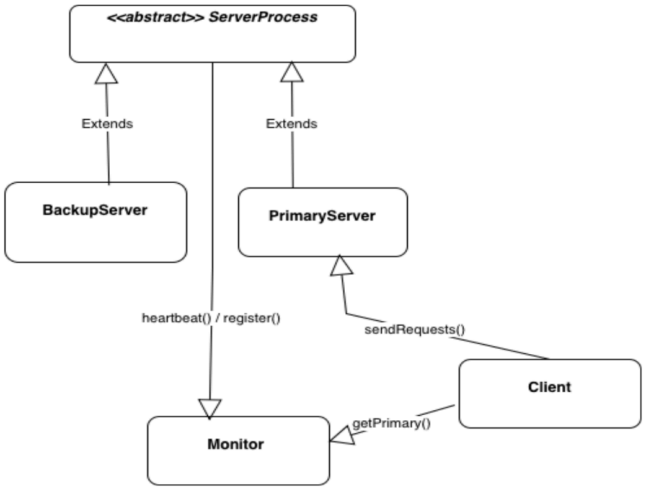
- The system logs important events like server failures, role changes, and client requests for easier monitoring and debugging.

Non-Functional Requirements (NFR)

- The system should detect failures quickly using heartbeat timeouts.
- Service should resume soon after failover, and the client should reconnect automatically.

UML Artifacts *(see full detailed diagrams in docs folder)*

To explain the design of our SRMS system, we created several UML diagrams that show both the structure and the flow of the system. The class diagram gives an overview of the main classes such as ServerProcess, PrimaryServer, BackupServer, the Monitor and the Client and how they are connected. We also included sequence diagrams to show how the system behaves in different situations. One diagram explains what happens during startup, where servers register with the Monitor and one is selected as the Primary. Another diagram shows the normal client request process, where the Client contacts the Monitor, finds the Primary server, and sends requests successfully. Finally, we created a failover sequence diagram to show what happens if the Primary server crashes, how a Backup server is promoted, and how the Client reconnects so the service can continue without major interruption.



Design Rationale and Class Responsibilities

This project is designed to keep the services running even if one fails. The main goal was to separate things like networking and thread management from the actual logic of how a server behaves in a specific role. By using an abstract base class (ServerProcess), we created a shared foundation for all nodes while also defining how Primary and Backup servers handle incoming requests differently. To keep the system stable during a crash, we implemented a failover strategy in the FailoverManager. So, if the main server goes down, every other server and the monitor can quickly agree on who is next in line without any confusion. We also separated the "control" side of the system from the "data" side. By using a dedicated port for promotion commands, we ensured that a busy server won't miss an important instruction from the monitor just because it is flooded with client requests. Our project follows the Single Responsibility Principle by giving every class a narrow, well-defined job. The ServerProcess acts as the backbone, handling the server socket and thread pool. The ConnectionHandler is solely responsible for managing the I/O of individual client sessions. The actual logic is split between PrimaryServer, which processes requests, and BackupServer, which stays in a standby mode until it is promoted. To keep track of the system, we used a HeartbeatSenderUDP to show us some status updates without blocking the main program, while the Monitor class watches those heartbeats to detect failures. The logic for choosing a new leader is kept entirely within the FailoverManager and the Log class provides a safe way to record events from multiple sources into a single file. Finally, ServerMain ties everything together by reading the configuration and spinning up the necessary threads to get the server running.

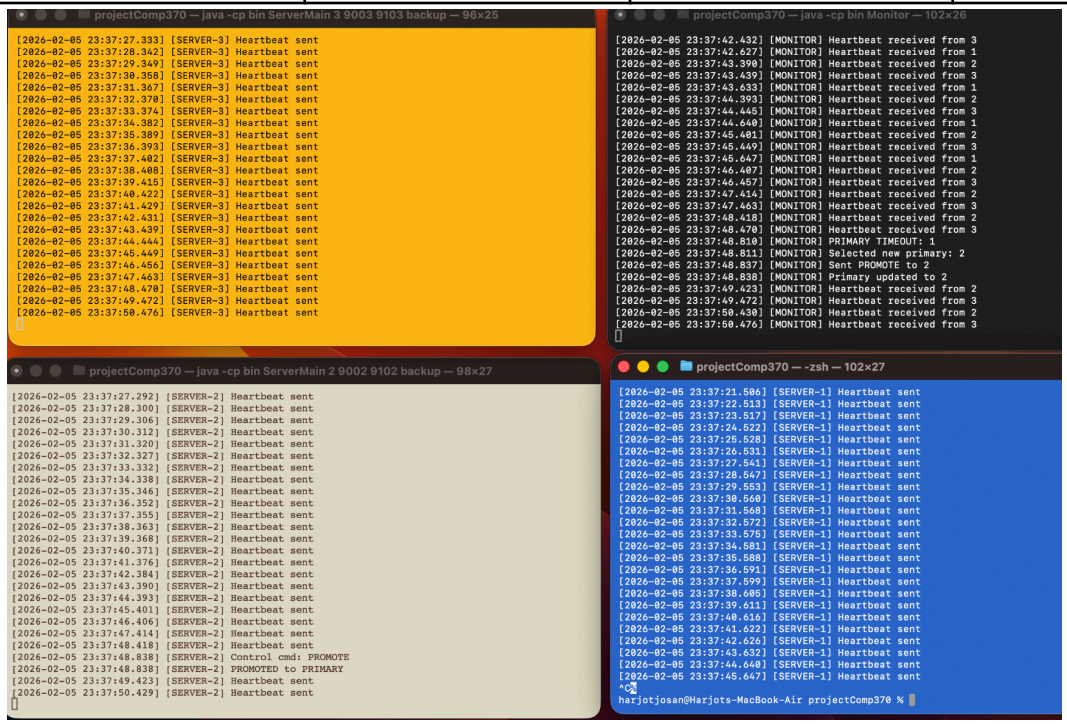
Description of State Replication Protocol and Election Algorithm

Our system uses a simple primary-backup setup. Only the Primary server handles client requests. The Backup servers stay ready in case the Primary fails. We don't copy full data between servers. The most important shared "state" is just which server is the Primary, and this is controlled by the Monitor. The election process is also handled by the Monitor. All servers send heartbeat messages. If the Monitor notices that the Primary has stopped sending heartbeats, it decides that the Primary has failed. To choose a new Primary, the Monitor uses a simple rule: the backup server with the lowest ID becomes the new Primary. This makes sure only one server is chosen and avoids problems where two servers might think they are in charge at the same time. After that, the Monitor tells the chosen server to become the new Primary and updates its records. When the Client asks for the Primary again, it gets the new address and reconnects automatically. This way, the system keeps running with only a short interruption and stays easy to understand and reliable.

Test scenarios and Results ***(see detailed Report with all test scenarios and timestamps under docs)***

After completing the java files, we compiled the files through commandline. For this we opened 4 terminals- one as the monitor, 3 as the servers (one main and 2 backups). To compile and find timestamps, we had to first start the monitor and type the command in it to start it followed by typing the other 3 commands in separate 3 terminals. Once they all connected they started sending heartbeats to the monitor.

Terminal 1- Monitor	**Server 1- Primary**	**Server 2- backup**	**Server 3**
cd ~/Desktop/projectComp370 java -cp bin Monitor	cd ~/Desktop/projectComp370 java -cp bin ServerMain 1 9001 9101 primary	cd ~/Desktop/projectComp370 java -cp bin ServerMain 2 9002 9102 backup	cd ~/Desktop/projectComp370 java -cp bin ServerMain 3 9003 9103 backup



This screenshot shows how all the servers were sending heartbeat to the monitor before we stopped Server 1 (the primary). This resulted in monitors promoting server 2 as the primary now. The results were:

- **Time of event:** [2026-02-05 23:37:45.647] [SERVER-1] Heartbeat sent
- **Detection time:** [2026-02-05 23:37:48.810] [MONITOR] PRIMARY TIMEOUT: 1

- 3.163 seconds
- **Failover time:** 0.028 seconds
  - Failure detection: [2026-02-05 23:37:48.810]
  - New primary server: [2026-02-05 23:37:48.838] [SERVER-2] PROMOTED to PRIMARY
- **State consistency:** Consistent – backup promoted cleanly

If all the servers are stopped then the monitor stops receiving heartbeat unless we start servers all over again. File **"Step 7 and 8.pdf"** has many more test scenarios and results in more detail and covers different tests done to show how the functionality of the project was successful.

Challenges Faced

When building a project like this the biggest problem we had was that things do not always happen in the order you think they will. One of the challenges we had was managing concurrency and thread safety in the distributed system. The program has a lot of things going on at the same time and every server and monitor in the system are running multiple threads and they're handling heartbeats, listening for client requests and monitoring timeouts all at the same time. So we had to be very careful with the shared data in the project to avoid conditions that could crash the program or lead to any inconsistencies. Another challenge was to make sure the network was always working right and so when we tested it on our computers, we had to make some changes to the network reliability and timing so that the system would not get confused and switch to a backup system for no good reason or because of some lag or error in code. We also had to work a lot on what to do when something went wrong with the network and how to fix it without making things worse. The network reliability and timing was very important to get right. Finally, we had some trouble with time management as with other heavy workloads from our classes and with midterms and projects due around the same time, we needed to find the right balance in order to finish on time. Despite these obstacles, we overcame them through communicating regularly through WhatsApp, debugging the project together, and gradually refining our rules until the final project behaved as expected. In the end we were able to achieve all the requirements of the project and we solved any problems encountered during the project effectively by working together in an organized manner.

Lessons Learned and Future Improvements

From this project, we learned how backup servers can help keep a system running when one server fails. We also learned that working with networking and multiple servers at the same time can be quite difficult and testing server crashes helped us understand how important quick failover logic also is. We also learnt how failover systems work and how important threads and networking are in a distributed system, and we also learnt the importance of timing issues and crashes so careful design and testing are very important. In the future, this system could be improved by adding real state replication between servers, using a better election method, and making failure detection more reliable to avoid false failovers.

Team Responsibilities and Contributions Breakdown

The project was completed collaboratively, with each member contributing to their assigned task while assisting others throughout development. All members helped update the README, tested the final project code, prepared documentation and helped write the report to ensure consistency. For the individual task breakdown, each group member was responsible for different steps throughout the project. Charvi was responsible for 'Requirements Gathering and Specification' and for the 'Use Cases and UML'. Simran was responsible for initial code and GitLab setup and 'System and OOP Design' and initial 'Networking and Concurrency Implementation'. Mohammad was responsible for 'Failover Logic and Election' and 'Client Implementation and Discovery'. Harjot was responsible for 'Logging, Monitoring, and Admin Interface' and 'Testing and Failure Scenarios'. The 'Documentation and Report' was completed by all group members.

References

Del Nero, R. (2023, December 6). When to use abstract classes vs. interfaces in Java. *InfoWorld*. https://www.infoworld.com/article/2171958/when-to-use-abstract-classes-vs-interfaces-in-java.html

draw.io - free flowchart maker and diagrams online. (n.d.). https://app.diagrams.net/

Heartbeat mechanism - Java Dynamic Management Kit 5.0 Tutorial. (n.d.). Oracle. Retrieved February 8, 2026, from https://docs.oracle.com/cd/E19206-01/816-4178/6madjde6e/index.html

Iluwatar. (n.d.). *Leader election pattern in Java: Mastering node coordination and consensus*. Java Design Patterns. https://java-design-patterns.com/patterns/leader-election/

Leader election in distributed systems, Amazon Builders' Library. (n.d.). Amazon Web Services, Inc. https://aws.amazon.com/builders-library/leader-election-in-distributed-systems/

Meghanathan, N. & Jackson State University. (n.d.). *A tutorial on socket programming in Java*. https://www.jsums.edu/nmeghanathan/files/2019/01/CSC435-Sp2019-Module-2-Socket-Programming-in-Java.pdf

User. (2015, June 24). *How to determine if a class meets the single responsibility principle?* Software Engineering Stack Exchange. https://softwareengineering.stackexchange.com/questions/154723/how-to-determine-if-a-class-meets-the-single-responsibility-principle

OpenAI ChatGPT - used for initial brainstorming of what is needed to be completed in project for improving project clarity and for explanations of how to overcome certain errors or for any questions we had about the project requirements (an example query screenshot is given below)

